Lista de exercícios – Arquivos em Python

Orientações para resolução da lista

- Esta lista trabalha exclusivamente o conteúdo de **arquivos texto**: a função `open`, os modos `"r"`, `"w"` e `"a"`, a escrita com `write`, o fechamento com `close` ou com `with`, a leitura linha a linha e os formatos de arquivo estudados (um dado por linha, campos separados por vírgula e JSON).
- Todas as questões são de **construção de código**. Antes de implementar qualquer solução, procure identificar claramente as entradas, o processamento necessário e as saídas esperadas.
- Todo código que manipula arquivos segue o mesmo ciclo: abrir, processar e fechar. Nas suas soluções, utilize sempre `with` e informe `encoding="utf-8"`.
- Cuidado com o modo `"w"`: ele apaga todo o conteúdo anterior do arquivo. Para acrescentar informações sem perder as existentes, utilize o modo `"a"`.
- A função `write` só aceita strings e não pula linha sozinha. Para separar os dados em linhas, é preciso escrever o caractere `"\n"`.
- Tudo o que vem de um arquivo de texto chega como string, e cada linha lida inclui o `"\n"` do final. Utilize `strip()` para limpar a linha e `int()` ou `float()` quando precisar de números.
- Para testar os programas que leem arquivos, crie os arquivos de entrada em um editor de texto, com o conteúdo dos exemplos, na mesma pasta do seu programa. Depois de cada execução, abra os arquivos gravados e confira o resultado.
- Desenvolva as soluções individualmente. O objetivo da lista é consolidar seu aprendizado e desenvolver autonomia na construção de programas.

Exercícios

1. Faça um programa que leia 5 números inteiros digitados pelo usuário e os armazene em uma lista. Em seguida, o programa deve gravar esses valores no arquivo `valores.txt`, na ordem inversa da leitura, com um valor por linha.

Depois de encerrar a gravação, o programa deve abrir o mesmo arquivo para leitura, recuperar os valores e exibir a soma deles.

Por exemplo, para as entradas 1, 2, 3, 4 e 5, o arquivo `valores.txt` deve conter:

```
5
4
3
2
1
```

e o programa deve exibir `Soma: 15`. A soma deve ser calculada a partir dos valores lidos do arquivo, e não da lista original.

2. O arquivo `temperaturas.txt` armazena as temperaturas registradas ao longo de um dia, com um valor real por linha. O arquivo pode conter linhas vazias, que devem ser ignoradas.

Por exemplo:

```
21.5
24.0

30.5
18.0
```

Faça um programa que leia o arquivo e exiba a quantidade de temperaturas registradas, a média, a maior e a menor temperatura.

Para o exemplo acima, a saída deve indicar:

```
Registros: 4
Media: 23.5
Maior: 30.5
Menor: 18.0
```

Caso o arquivo não possua nenhuma temperatura, exiba apenas a mensagem `Nenhum registro encontrado`.

3. Crie uma função chamada `numerar_linhas(origem, destino)` que receba os nomes de dois arquivos. A função deve ler o arquivo de origem e gravar, no arquivo de destino, cada uma de suas linhas precedida pelo seu número, seguido de dois-pontos e de um espaço. A numeração começa em 1. Ao final, a função deve retornar a quantidade de linhas gravadas.

Por exemplo, se o arquivo de origem possuir o conteúdo:

```
comprar pao
estudar arquivos
lavar o carro
```

o arquivo de destino deve ficar assim:

```
1: comprar pao
2: estudar arquivos
3: lavar o carro
```

No programa principal, chame a função para os arquivos `tarefas.txt` e `numeradas.txt` e exiba a quantidade retornada.

4. Uma papelaria guarda seu estoque no arquivo `estoque.txt`, no qual cada linha contém o nome do produto e a quantidade, separados por vírgula:

```
Caneta,15
Borracha,25
Lapis,50
```

Implemente duas funções:

- `carregar_estoque(nome_arquivo)`, que lê o arquivo e retorna um dicionário no qual a chave é o nome do produto e o valor é a quantidade, já convertida para número inteiro;
- `salvar_estoque(estoque, nome_arquivo)`, que recebe um dicionário e grava seu conteúdo no arquivo, um produto por linha, no mesmo formato.

No programa principal, carregue o estoque, leia pelo teclado o nome de um produto e uma quantidade recebida do fornecedor e atualize o dicionário. Se o produto já existir, a quantidade deve ser somada; caso contrário, o produto deve ser criado. Por fim, salve o estoque atualizado no mesmo arquivo.

Por exemplo, após o recebimento de 10 unidades de `Caneta` e, em outra execução, de 8 unidades de `Regua`, o arquivo deve conter:

```
Caneta,25
Borracha,25
Lapis,50
Regua,8
```

5. Crie uma função chamada `salvar_json(dados, nome_arquivo)` que receba um dicionário, no qual as chaves são strings e os valores são números inteiros, e grave seu conteúdo em um arquivo no formato JSON, com um item por linha.

Por exemplo, para o dicionário:

```
{"Ana": 12, "Bia": 7, "Caio": 20}
```

o arquivo gravado deve ter exatamente o seguinte conteúdo:

```
{
    "Ana": 12,
    "Bia": 7,
    "Caio": 20
}
```

Observe que as chaves são gravadas entre aspas duplas, que cada item é precedido por quatro espaços e que não existe vírgula depois do último item. Para um dicionário vazio, o arquivo deve conter apenas a linha com `{` e a linha com `}`.

No programa principal, leia um número inteiro N e, em seguida, o nome e a pontuação de N jogadores, armazenando-os em um dicionário. Por fim, utilize a função para gravar o arquivo `placar.json`.

6. Faça um programa para controlar a lista de convidados de um evento. O programa deve ler nomes digitados pelo usuário até que ele digite `fim` e gravar cada nome em uma linha do arquivo `convidados.txt`.

Os nomes informados em uma execução devem ser **acrescentados** ao arquivo, sem apagar os nomes gravados em execuções anteriores.

Depois de encerrar a gravação, o programa deve abrir o arquivo para leitura e exibir a quantidade total de convidados registrados e o nome mais longo presente no arquivo. Em caso de empate, considere o primeiro nome encontrado.

Por exemplo, se o arquivo já possuía os nomes `Ana` e `Bernardo` e o usuário digitar `Caio` e `fim`, a saída deve indicar:

```
Convidados: 3
Nome mais longo: Bernardo
```

7. Implemente duas funções para armazenar e recuperar listas de números inteiros.

A função `salvar_lista(valores, nome_arquivo)` deve receber uma lista de números inteiros e o nome de um arquivo e gravar um elemento por linha. Se o arquivo já existir, seu conteúdo anterior deve ser substituído.

A função `carregar_lista(nome_arquivo)` deve receber o nome de um arquivo e retornar uma lista com os valores armazenados, já convertidos para números inteiros. Linhas vazias devem ser ignoradas.

No programa principal, leia N números inteiros, salve a lista no arquivo `numeros.txt`, carregue o arquivo em uma segunda lista e verifique, elemento por elemento, se o resultado é igual à lista original. Exiba as duas listas e a mensagem `Listas iguais` ou `Listas diferentes`.

8. O arquivo `notas.txt` contém, em cada linha, o nome de um aluno e a sua nota, separados por vírgula:

```
Ana,8.5
Bruno,4.0
Carla,6.0
Davi,5.5
Eva,9.0
```

Faça um programa que leia o arquivo e calcule a média das notas da turma. Em seguida, o programa deve gravar no arquivo `aprovados.txt` apenas os alunos com nota maior ou igual a 6 e, no arquivo `recuperacao.txt`, os demais alunos. Os dois arquivos devem manter o mesmo formato do arquivo original.

Ao final, exiba a média da turma, a quantidade de alunos gravada em cada arquivo e o nome do aluno com a maior nota.

Para o exemplo acima, a média é 6.6, o arquivo `aprovados.txt` recebe três alunos, o arquivo `recuperacao.txt` recebe dois e a maior nota pertence a Eva.

9. Uma loja registra cada venda realizada em uma linha do arquivo `vendas.txt`. Cada linha contém o nome do produto, a quantidade vendida e o preço unitário, separados por vírgula. Um mesmo produto pode aparecer várias vezes no arquivo.

Por exemplo:

```
Caneta,10,2.5
Caderno,2,18.0
Caneta,4,2.5
Lapis,20,1.0
Caderno,1,18.0
```

Faça um programa que leia o arquivo e utilize um dicionário para acumular, para cada produto, o valor total arrecadado, considerando que o valor de cada venda corresponde à quantidade multiplicada pelo preço unitário.

Em seguida, o programa deve gravar o arquivo `resumo.txt` com uma linha para cada produto, no formato `produto,total`, na ordem em que os produtos aparecem pela primeira vez no arquivo de vendas.

Para o exemplo acima, o arquivo `resumo.txt` deve conter:

```
Caneta,35.0
Caderno,54.0
Lapis,20.0
```

Ao final, exiba na tela o produto com o maior valor arrecadado e o total geral de vendas da loja. Em caso de empate, considere o primeiro produto encontrado.

10. Um sistema de controle de estoque armazena seus dados no arquivo `estoque.json`, no formato JSON, com um produto por linha:

```
{
    "Caderno": 10,
    "Borracha": 25,
    "Lapis": 50
}
```

As movimentações do dia são registradas no arquivo `movimentos.txt`. Cada linha contém o nome de um produto e uma quantidade, separados por vírgula. Quantidades positivas representam entradas e quantidades negativas representam saídas:

```
Caderno,-4
Lapis,-60
Caneta,30
Borracha,-25
Caderno,2
```

Faça um programa que:

a) leia o arquivo `estoque.json` e reconstrua o dicionário de produtos e quantidades, convertendo as quantidades para números inteiros;

b) leia o arquivo `movimentos.txt` e aplique cada movimentação ao dicionário, na ordem em que aparecem;

c) grave novamente o arquivo `estoque.json` com os dados atualizados, respeitando o formato JSON.

Durante o processamento das movimentações, considere as seguintes regras:

- uma entrada de um produto que ainda não existe no estoque deve criar esse produto;
- uma saída de um produto que não existe, ou em quantidade maior que a disponível, deve ser recusada. Nesse caso, o estoque não é alterado e a linha recusada deve ser **acrescentada** ao arquivo `recusados.txt`, sem apagar registros de dias anteriores;
- um produto cuja quantidade chegar a zero deve ser removido do dicionário.

Para o exemplo acima, a saída de 60 unidades de Lapis é recusada, e o arquivo `estoque.json` deve terminar com o seguinte conteúdo:

```
{
    "Caderno": 8,
    "Lapis": 50,
    "Caneta": 30
}
```

Lembre-se de que, no formato JSON, não existe vírgula depois do último item. Ao final, exiba na tela a quantidade de movimentações aplicadas e a quantidade de movimentações recusadas.